

AI224: Operating Systems — Lecture 13

Dr. Karim Lounis

Jan - May 2025



Message Passing: Interprocess Communication

InterProcess Communication

Definition

InterProcess communication (IPC). Is the topic which deals with how processes communicate with each other for collaboration (cooperation).

Mainly, there are three issues in interprocess communication:

1. How a process can pass information to another process.

e.g., a process needs to communicate some computing results to another process

2. How to avoid that two processes do not get in each other's way.

e.g., two processes trying to grab the last available flight seat.

3. How to enforce sequencing between processes.

e.g., a process needs to wait for another so that it can start.

InterProcess Communication

Mainly, there are three issues in interprocess communication:

1. How a process can pass information to another one.

e.g., threads can share variables, share a file, use pipes, primitive signals

2. How to avoid that two processes do not get in each other's way.

We use semaphores or monitors for mutual exclusion

3. How to enforce sequencing between processes.

We use semaphores or monitors with condition variables for ordering

InterProcess Communication

Mainly, there are three issues in interprocess communication:

1. How a process can pass information to another one.

e.g., threads can share variables, share a file, use pipes, primitive signals

2. How to avoid that two processes do not get in each other's way.

We use semaphores or monitors for mutual exclusion

3. How to enforce sequencing between processes.

We use semaphores or monitors with condition variables for ordering

Issue. These mechanisms (semaphores, monitors, shared-variables, ...) have been designed to solve the above issues when processes are executed on the same motherboard (share CPU and memory).

Message passing

Definition

Message passing. Is a mechanism that allows two or more processes running on same or different computers to communicate and collaborate through a telecommunication network (e.g., loopback, LAN, WLAN, ...).

This method of interprocess communication uses two fundamental primitives implemented as system calls:

1. `Send()`. Allows a process to send a message to another process thru the network by specifying the destination address.

e.g., `Send(destination, message);`

2. `Receive()`. Allows a process to receive a message from a particular process thru the network by specifying the source address (or ANY).

e.g., `Receive(source, message);`

Overview of Message passing



Issues in Message passing

When using message passing, other types of issues arise. Those issues are not found in semaphores, monitors, or any local data sharing mechanism.

Message loss. In a communication network, messages tend to be lost, additional mechanisms should be used to used such as retransmission and acknowledgment.

Message duplication. Messages may get duplication at the reception e.g., an acknowledgment was lost. Usually, sequence numbers are used.

Transfer delays. In a communication network, the arrival time of messages to their destinations is non-deterministic (orders not preserved).

Error detection. Some bits inside the message may get flipped.

Processes identification. In a communication network, processes need to have a sort of identity to be identified and located in a network.

Security. Authentication, confidentiality, integrity, availability, non repudiation, etc.

Process identification in Message passing

Process identification or naming focus on how processes refer to each other so that they can communicate.

1. Direct communication: Assign each process a unique address (identifier: e.g., PID) and have messages be addressed to processes.

```
Send(P, message);
```

```
Receive(Q, message);
```

or

```
Send(1, message);
```

```
Receive(3, message);
```

2. Indirect communication: Use of a **mailbox**. A data structure of size N, to buffer messages (Many-to-one).

```
Send(MBx, message);
```

```
Receive(MBx, message);
```

Type of Message Passing

Message passing may be either blocking or nonblocking (implementation style for `send()` & `receive()`):

Blocking. A.k.a., synchronous message passing.

- *Blocking send:* The sender process is blocked while the message that it has sent has not been received (e.g., **mailbox full**).
- *Blocking receive:* The receiver process is blocked till it receives a message.

Nonblocking. A.k.a., Asynchronous message passing.

- *Nonblocking send:* The sender process send the message and continue computing (**usually there is a buffer at the reception**).
- *Nonblocking receive:* The receiver receives a valid message or null (**reads from the buffer**).

Message passing example

When both `send()` and `receive()` are blocking, the mechanism is called **rendezvous**.

We can use **rendezvous** to easily solve the producer-consumer problem:

Producer

```
Object item;
Message m;
While(true)
{
    <Produce item>
    <Put item inside m>
    Send(m);
}
```

Consumer

```
Object item;
Message m;
While(true)
{
    Receive(m);
    <Get item from m>
    <Consume item>
}
```

You can use the standard C library **MPI (Message Passing Interface)**.

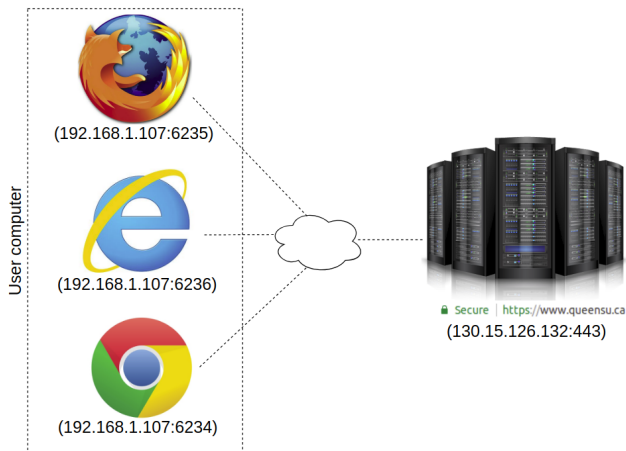
Sockets

Definition

Sockets. Are endpoint communication objects used to establish connections between two processes in order to communicate over the network or within a localhost.

- A socket is tuple (a, b) s.t. a is an IP address (e.g., 192.168.1.3) and b is a communication port number (e.g., 1234). Usually denoted $(a:b)$.
- A pair of processes communicating over a network employs a pair of sockets: $(192.168.1.10:1255) \leftrightarrow (130.15.126.132:80)$.
- Use a client-server architecture.
- The server (server socket) waits for incoming connections from clients on a specific port (80 HTTP server, 23 Telnet, 22 SSH, 53 DNS, ...).
- The client (client socket) initiates a request for a connection with a server. It gets assigned a port number by its host computer ($p > 1024$).

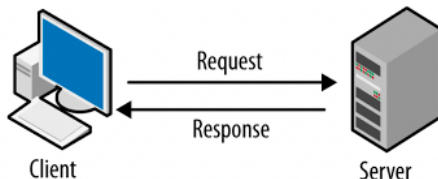
Sockets



All connections consists of a unique pair of sockets.

Sockets

Socket adopt a **client-server** communication model.



In Java, you can use **Java sockets** to make processes communicate, either locally through the operating systems's loopback, or through a network.

You can check this [java socket-based application](#) (Lab 13). Visualize the source code and make sure to understand the mechanism of **Java sockets**.

The provided application uses TCP sockets (Socket and ServerSocket). You can also use UDP sockets (DatagramSocket).

[A quick demos...](#)

Remote Procedure Calls

Definition

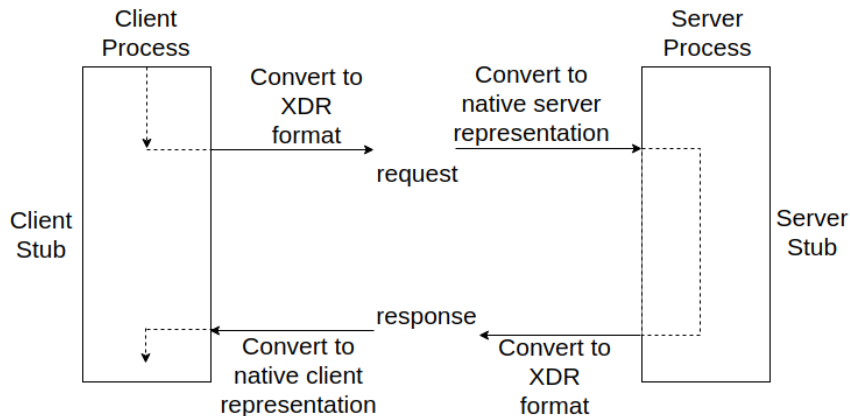
RPC. Is a communication concept that consists of allowing a process running on a computer A, to remotely execute a procedure by a process running on another computer B, in the same way as if it occurred on A.

- A server has to implement a set of procedures and waits on a particular port for incoming procedure calls.
- Clients initiate connections with the server and send messages that contain the name of the procedure to execute as well as the parameters.

Packaging the parameter in a form that can be transmitted over the network is called marshaling: includes transforming machine-dependent data into XDR (eXternal Data Representation).

- In Java, RPCs are implemented using RMI (Remote Method Invocation).

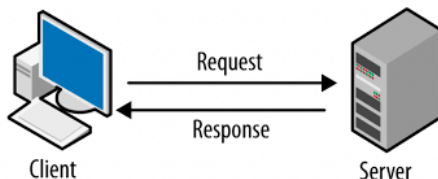
Remote Procedure Calls



Overview of RPC (Remote Procedure Calls).

RPC

RPCs adopt a **client-server** communication model.



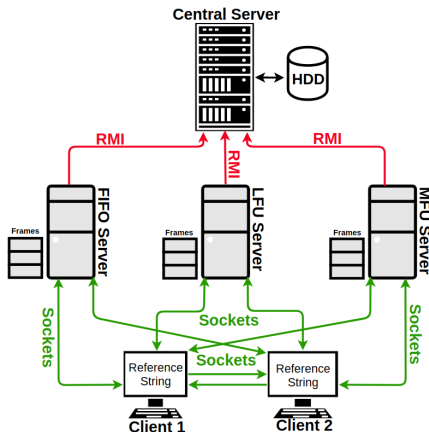
In Java, you can use **Java RMI** to make processes communicate, either locally through the operating systems's loopback, or through a network.

You can check this [squaring java RMI-based application](#) (Lab 14). Visualize the source code and make sure to understand the mechanism of **Java RMI**.

A quick demos...

Java Sockets and RMI (Summer Lab)

Consider the following network architecture and configuration (files):



Use the following reference string:

7, 0, 1, 2, 4, 3, 0, 6, 2, 3, 0, 5, 2, 1, 0, 5, 1, 7, 0, 2

Operating System Security

Overview: What is an Operating System?

Definition

It is a set of programs (called **modules**) that cooperate for the good management and use of **computer resources**. These modules can be classified into two classes, primary (kernel) and secondary.

• Primary modules (Kernel):

- Process manager module.
- Memory manager module.
- File system module.
- Interrupt handling module.
- Other?

• Secondary modules:

- Drivers.
- Networking module.
- Security manger module.
- The Shell (cmd, gui, both)
- Others?

✚ There are various types of operating systems: real-time, mobile, single-taking, multi-tasking, embedded, distributed, network-based, etc.

✚ Some are open-source, whereas other are closed-source operating systems.

✚ Some operating systems provide much better security than others.

Overview: Operating Systems Security

Definition

Operating Systems Security. Is the discipline that studies how to secure operating systems from security threats.

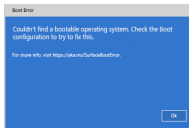
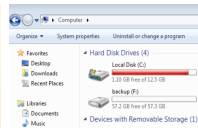
Why do we need a secure operating system, or to secure our operating system?

- Operating systems often store and manage critical data, including user **credential** **financial information**, and **personal files**. **Strong OS security safeguards this data from unauthorized access or modification.**
- Security threats can disrupt **system functionality**, leading to crashes, data loss, or denial-of-service attacks. **Robust OS security helps ensure system stability, availability, and automatic fault-recovery.**
- A well-secured operating system minimizes the risk of malware infections, unauthorized access attempts, and other security incidents.

Overview: Operating Systems Security Aspect

Operating System Aspect. Securing operating systems is crucial too:

- Install regular updates and security patches, & disable unnecessary services.
- Secure user accounts with username, password, and access rights.
- Install anti-malware (e.g., anti-spyware and anti-viruses).
- Implement strong authentication (as per the standard guidelines).
- Audit and log events to track and monitor users' activities.
- Use security software such as firewalls to protect from outsiders.
- Backup and recovery plan — incident response plan.
- Never use cracked operating systems or software. There is no liability.
- Do not plug unverified and untrusted devices (e.g., USB, power cable, etc) that you find abandoned or been given during a social event.



Overview: Operating Systems Security Aspect

Operating System Aspect. Securing operating systems is crucial too:

- Install regular updates and security patches. & disable unnecessary services.

Operating systems, often downloaded from unofficial sources or untrustworthy websites, were later found to be malicious.

Linux Mint Website Hacked; ISO Downloads Replaced with a Backdoor

22 février 2016



The systems of users who downloaded Linux Mint on February 20 may be at risk after it was discovered that Hackers from Sofia, Bulgaria managed to hack into Linux Mint, currently **one of the most popular Linux distributions** available. According to Linux Mint's report, the hacker tricked users into downloading a version of Linux Mint ISO with a backdoor installed by replacing the download links on the site. The link leads to one of their servers offering malicious ISO images of the Linux Mint 17.3 Cinnamon edition. The website has been down since February 21, Sunday, resulting in the loss of thousands of downloads.



Ubuntu Spyware: What to Do?

by [Richard Stallman](#)

Since [Ubuntu version 16.04](#), the spyware search facility is now disabled by default. It appears that the campaign of pressure launched by this article has been partly successful. Nonetheless, offering the spyware search facility as an option is still a problem, as explained below. Ubuntu should make the network search a command users can execute from time to time, not a semipermanent option for users to enable (and probably forget).

Even though the factual situation described in the rest of this page has partly changed, the page is still important. This example should teach our community not to do such things again, but in order for that to happen, we must continue to talk about it.

One of the major advantages of free software is that the community protects users from malicious software. Now Ubuntu [GNU/Linux](#) has become a counterexample. What should we do?

Proprietary software is associated with malicious treatment of the user: surveillance code, digital handcuffs (DRM or Digital Restrictions Management) to restrict users, and back doors that can do nasty things under remote control. Programs that do any of these things are malware and should be treated as such. Widely used

- End.